



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

*Приложение для визуализации световых
паттернов гирлянд*

Студент	<u>ИУ7-56Б</u>	<u></u>	<u>И.Д. Журавлев</u>
	(группа)	(подпись, дата)	(И.О. Фамилия)
Руководитель курсовой работы		<u></u>	<u>А.Ю. Быстрицкая</u>
		(подпись, дата)	(И.О. Фамилия)
Консультант		<u></u>	<u></u>
		(подпись, дата)	(И.О. Фамилия)

РЕФЕРАТ

Расчётно-пояснительная записка объёмом 44 с., в том числе 14 рис., 4 табл., 20 источников, 2 прил.

МОДЕЛИРОВАНИЕ ОСВЕЩЕНИЯ, ТРАССИРОВКА ЛУЧЕЙ, ПРЕЛОМЛЕНИЕ СВЕТА, КАРТЫ ОСВЕЩЕННОСТИ, КАУСТИКИ, ЗАКОН СНЕЛЛИУСА, АППРОКСИМАЦИЯ ФРЕНЕЛЯ, МОДЕЛЬ ЛАМБЕРТА.

Разработана программная система для физически корректного моделирования световых паттернов гирлянд с учетом двукратного преломления в защитных колпачках (сферический слой из двух концентрических сфер). Реализован программный комплекс на языках C++ и Python с использованием `pybind11`, обеспечивающий трассировку лучей с учетом преломления (закон Снеллиуса, аппроксимация Френеля), формирование карт освещенности для диффузных поверхностей (модель Ламберта), поддержку стохастического рассеивания. Графический интерфейс на Tkinter предоставляет инструменты создания паттернов источников света и интерактивное управление камерой.

В исследовательской части показано, что вычислительная сложность алгоритма составляет $O(N \cdot M)$, где N — число источников, M — число объектов. Время расчета линейно зависит от количества источников (0.36 сек/источник) и объектов (0.35 сек/объект), слабо зависит от разрешения карт (прирост 11.5% при увеличении разрешения в 5 раз). Лимитирующим фактором производительности является этап генерации лучей (80% времени), в то время как запись в текстурные атласы вносит менее 12% задержки.

Областью применения являются предварительная визуализация декоративного освещения в архитектурном дизайне, планирование световых инсталляций и образовательные приложения для демонстрации принципов геометрической оптики.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
ВВЕДЕНИЕ	7
1 Аналитическая часть	8
1.1 Анализ предметной области	8
1.1.1 Трассировка лучей как метод моделирования света	8
1.1.2 Предварительный расчет освещения	9
1.2 Обзор существующих аналогов	9
1.2.1 DIALux evo	9
1.2.2 Blender (Cycles)	9
1.2.3 Unreal Engine 5	9
1.2.4 Mitsuba 3	9
1.2.5 LuxCoreRender	9
1.2.6 Сравнительный анализ аналогов	10
1.3 Постановка задачи	10
1.3.1 IDEF0 диаграмма	11
2 Конструкторская часть	12
2.1 Разработка архитектуры системы	12
2.1.1 IDEF0 диаграмма первого уровня (A0)	12
2.2 Математическое описание	12
2.2.1 Модель освещения (Закон Ламберта)	12
2.2.2 Модель преломления (Закон Снеллиуса)	13
2.2.3 Аппроксимация Френеля (формула Шлика)	14
2.2.4 Модель поглощения и фильтрации света	14
2.2.5 Модель рассеивания (матовость)	14
2.2.6 Модель пересечения луча со сферой	15
2.2.7 Модель пересечения луча с плоскостью (стена)	15
2.2.8 Модель распределения направлений лучей	16
2.2.9 Схемы основных алгоритмов	17
2.3 Проектирование интерфейса (UI/UX)	22
2.3.1 Описание взаимодействия пользователя с программой	22
2.3.2 Структура интерфейса	23
3 Технологическая часть	24
3.1 Выбор средств реализации	24
3.2 Структура программного комплекса	24

3.2.1	Общая архитектура системы	25
3.2.2	Диаграмма компонентов (UML)	25
3.2.3	Описание назначения основных классов и функций	26
3.2.4	Модели данных	26
3.3	Особенности реализации	27
3.4	Тестирование корректности	28
3.4.1	Методы верификации ПО	28
3.4.2	Примеры работы программы на разных данных	28
3.5	Руководство пользователя	29
3.5.1	Интерфейс программы	29
3.5.2	Основные возможности программы	31
4	Исследовательская часть	32
4.1	Методика тестирования производительности	32
4.2	Результаты тестирования производительности	32
	ЗАКЛЮЧЕНИЕ	36
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37
	ПРИЛОЖЕНИЕ А	39
A	Листинги кода	39
A.1	Функция расчета преломления луча	39
A.2	Функция аппроксимации Френеля	39
A.3	Обработка преломления в колпачке	39
A.4	Ядро трассировки лучей	41
	ПРИЛОЖЕНИЕ Б	44

ВВЕДЕНИЕ

Цель работы — создание программной системы для моделирования световых паттернов статических декоративных гирлянд, обеспечивающей физически обоснованный расчет распределения света и интерактивную визуализацию результатов.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) разработать математическую модель распространения света от точечных источников с учетом преломления в защитных колпачках и реализовать алгоритм трассировки лучей для расчета распределения освещенности на диффузных поверхностях;
- 2) создать систему предварительного расчета освещения с формированием карт освещенности и поддержкой стохастического рассеивания для имитации матовых материалов колпачков;
- 3) разработать графический пользовательский интерфейс с инструментами создания паттернов гирлянды (точка, линия, дуга, круг) и редактирования параметров источников света и защитных колпачков;
- 4) реализовать интерактивное управление камерой и визуализацию результатов расчета в реальном времени на основе предрасчитанных карт освещенности;
- 5) провести анализ производительности системы и исследование зависимости времени расчета от различных параметров.

1 Аналитическая часть

1.1 Анализ предметной области

Моделирование световых эффектов гирлянды — задача визуализации распространения света от множественных точечных источников с учетом их расположения и оптических свойств защитных элементов. Основной визуальной характеристикой гирлянды является световой узор, формируемый на освещаемых поверхностях (стенах, предметах интерьера), что требует учета физических законов геометрической оптики.

Задача моделирования гирлянд специфична из-за малых размеров источников и каустик, формируемых преломлением света в защитных колпачках. Это требует высокого разрешения расчетов и точного моделирования преломления.

Система должна обеспечивать интерактивное размещение источников, настройку параметров защитных колпачков (материал, прозрачность) и визуализацию карты освещенности после предварительного расчета.

Область применения: проектирование декоративного освещения, создание виртуальных инсталляций и демонстрация принципов геометрической оптики.

Для моделирования световых эффектов гирлянды используется геометрическая оптика, описывающая распространение света как движение лучей, подчиняющихся законам отражения и преломления [1, 3]. Математические модели и формулы представлены в разделе 2.2.

1.1.1 Трассировка лучей как метод моделирования света

Трассировка лучей — метод моделирования распространения света путем отслеживания пути лучей от источников света до поверхностей или от камеры до источников света [2, 4]. Метод позволяет физически корректно моделировать преломление, отражение и формирование каустических эффектов.

Защитный колпачок источника света моделируется как сферический слой — две концентрические сферы с общим центром, но разными радиусами. Источник света находится внутри малой сферы, что обеспечивает двойное преломление луча: на входе в материал (внешняя сфера) и на выходе (внутренняя сфера).

Ограничения модели:

- не учитывается дисперсия света (зависимость показателя преломления от длины волны);
- не учитывается многократное внутреннее отражение в колпачке (только первое преломление на входе и выходе);
- используется упрощенная модель рассеивания (стохастическое возмущение вместо физически точных моделей микрорельефа);
- не учитывается поляризация света;
- расчет выполняется только для статической сцены (динамические эффекты не под-

держиваются);

— модель освещения ограничена диффузным отражением (модель Ламберта), зеркальные отражения не учитываются.

1.1.2 Предварительный расчет освещения

Предварительный расчет освещения — техника, при которой освещение рассчитывается один раз для статической сцены и сохраняется в текстуры (карты освещенности). Это позволяет достичь высокой производительности при интерактивном просмотре сцены, так как тяжелые вычисления выполняются заранее.

1.2 Обзор существующих аналогов

1.2.1 DIALux evo

Использует алгоритмы Radiosity и Ray Tracing для расчета освещенности [5]. Не предоставляет средств для процедурной генерации каустических эффектов от точечных источников с учетом преломления в защитных колпачках.

1.2.2 Blender (Cycles)

Использует алгоритм Path Tracing с методом Монте-Карло [6, 7]. Избыточная сложность настройки каустик для точечных источников с преломляющими элементами.

1.2.3 Unreal Engine 5

Система Lumen использует гибридный подход с трассировкой лучей в реальном времени [8]. Приоритет производительности над физической точностью, оптимизация для игровых сцен усложняет точное моделирование каустик от малых преломляющих элементов.

1.2.4 Mitsuba 3

Исследовательская система рендеринга, использует спектральный рендеринг [9]. Управляется через скрипты, лишена графического интерфейса для интерактивной расстановки источников света.

1.2.5 LuxCoreRender

Использует Bidirectional Path Tracing [10]. Низкая скорость работы (десятки минут для сложных сцен) делает невозможным интерактивную работу.

1.2.6 Сравнительный анализ аналогов

Сравнение систем проводится по критериям метода моделирования, специализации и возможности работы в режиме реального времени (после предварительного расчета).

Таблица 1.1 — Сравнительный анализ существующих решений

Система	Метод расчета	Специализация	Режим работы
DIALux evo	Radiosity / Ray Tracing	Инженерная светотехника	Интерактивный
Blender (Cycles)	Трассировка путей	3D-анимация	Предварительный расчет
Unreal Engine	Трассировка лучей / предрасчет	Игры	Реальное время
Mitsuba 3	Исследовательские алгоритмы	Научные исследования	Предварительный расчет
LuxCoreRender	Двунаправленная трассировка путей	Реалистичная графика	Предварительный расчет

Вывод: Существующие системы ориентированы на общие задачи визуализации, что усложняет оперативную разработку дизайна декоративного освещения.

1.3 Постановка задачи

Программа должна выполнять моделирование световых паттернов статических декоративных гирлянд с учетом физических законов геометрической оптики.

Входные данные: Конфигурация источников света (позиция, цвет, интенсивность, количество лучей), параметры защитных колпачков (радиусы, коэффициент преломления, прозрачность, рассеивание), геометрия сцены (стена, сферы), параметры камеры и расчета.

Выходные данные: Карты освещенности для стены и сфер (двумерные массивы RGB значений), изображение вида камеры.

Ограничения: Система визуализирует только эффекты освещения на поверхностях; расчет выполняется для статической конфигурации; количество источников и объектов ограничено производительностью; разрешение карт ограничено доступной памятью.

Требования к системе:

- моделирование прохождения света через преломляющие среды (колпачки);
- формирование карт освещенности на диффузных поверхностях (стенах и объектах сцены);
- поддержка стохастического рассеивания для имитации матовых материалов;
- обеспечение интерактивного просмотра сцены за счет технологии предварительного расчета освещения;
- графический пользовательский интерфейс для интерактивного размещения источников света и объектов сцены;

- инструменты создания паттернов гирлянды (точка, линия, дуга, круг);
- возможность редактирования параметров источников света (позиция, цвет, интенсивность) и защитных колпачков (коэффициент преломления, прозрачность, рассеивание);
- интерактивное управление камерой для просмотра результатов с разных ракурсов;
- визуализация результатов расчета в реальном времени.

1.3.1 IDEF0 диаграмма

На рисунке 1.1 представлена контекстная диаграмма системы моделирования световых паттернов гирлянды (диаграмма уровня A-0).

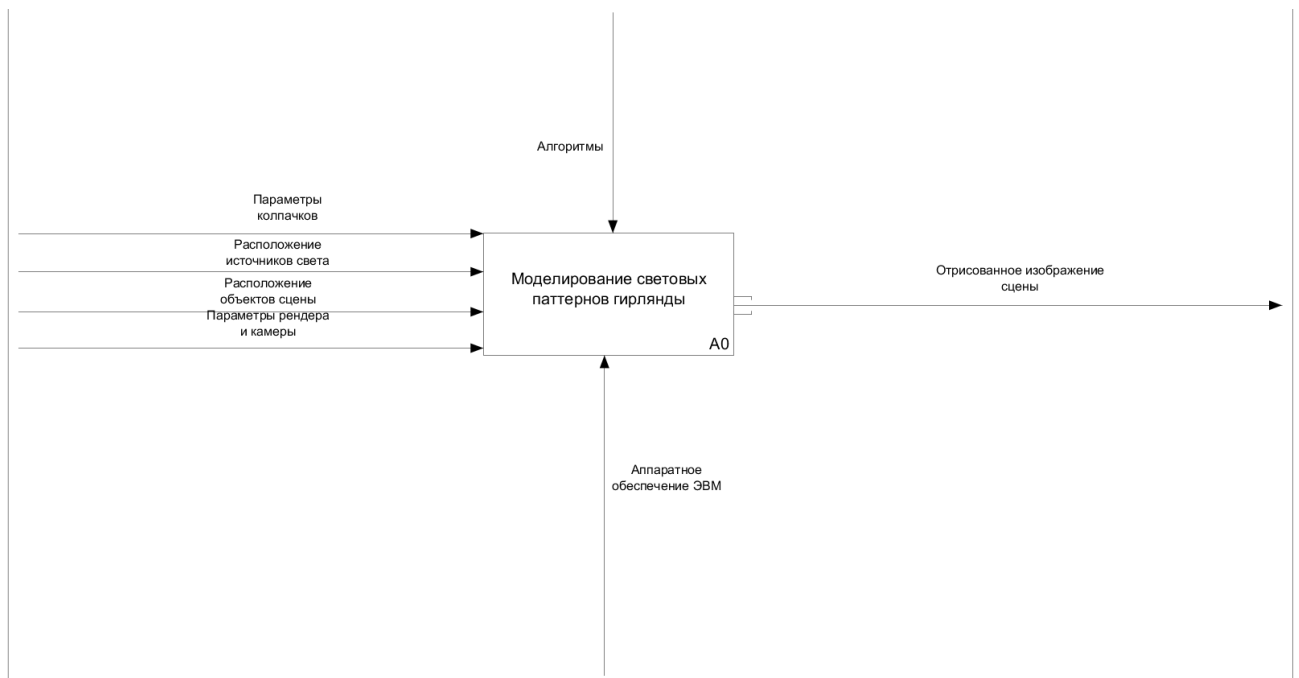


Рисунок 1.1 — Контекстная диаграмма системы симуляции (IDEF0 A-0)

2 Конструкторская часть

2.1 Разработка архитектуры системы

Модуль трассировки лучей инкапсулирует вызовы C++ ядра, передавая конфигурацию сцены в виде структур данных и возвращая RGB-массивы освещенности. Вычислительный модуль (C++) выполняет трассировку лучей от источников света, расчет преломления в колпачках, определение пересечений с объектами сцены и формирование карт освещенности. Модуль визуализации преобразует предрасчитанные карты освещенности в изображения для отображения.

2.1.1 IDEF0 диаграмма первого уровня (A0)

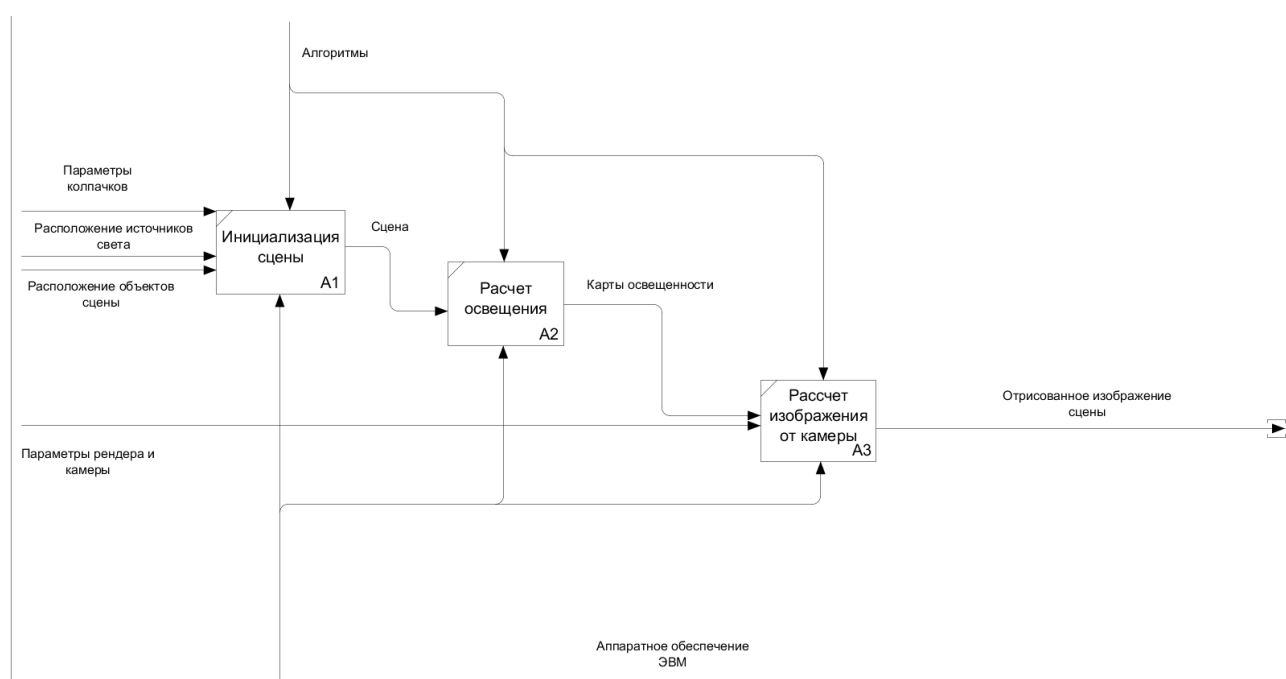


Рисунок 2.1 — Диаграмма декомпозиции первого уровня (IDEF0 A0)

2.2 Математическое описание

2.2.1 Модель освещения (Закон Ламберта)

Для расчета интенсивности освещения используется модель диффузного отражения Ламберта [3, 7]:

$$I = I_0 \cdot \rho \cdot \max(0, \vec{n} \cdot (-\vec{d})) \quad (2.1)$$

где:

- I — результирующая интенсивность;
- I_0 — интенсивность луча;

- ρ — коэффициент диффузного отражения (0–1);
- $\vec{n}$ — нормаль к поверхности;
- $\vec{d}$ — направление луча.

При накоплении освещенности от множества лучей:

$$I_{\text{total}} = \sum_{i=1}^N I_i \quad (2.2)$$

где:

- I_{total} — суммарная интенсивность освещения;
- N — количество лучей;
- I_i — интенсивность от i -го луча.

2.2.2 Модель преломления (Закон Снеллиуса)

Защитный колпачок источника света представляет собой сферический слой, состоящий из двух концентрических сфер с общим центром, но разными радиусами (внешняя и внутренняя сферы). Источник света расположен внутри малой сферы. При трассировке луча от источника происходит двойное преломление: сначала луч пересекает внешнюю сферу (вход в материал колпачка), затем внутреннюю сферу (выход из материала). Нормаль к поверхности в каждой точке пересечения вычисляется как нормаль к соответствующей сфере в этой точке.

При прохождении луча через границу раздела двух сред направление изменяется согласно закону Снеллиуса [3]:

$$n_1 \sin(\theta_1) = n_2 \sin(\theta_2) \quad (2.3)$$

где:

- n_1 — показатель преломления первой среды;
- n_2 — показатель преломления второй среды;
- θ_1 — угол падения;
- θ_2 — угол преломления.

Вектор преломленного луча:

$$\vec{d}_{\text{refr}} = \eta \vec{d} + (\eta \cos(\theta_1) - \cos(\theta_2)) \vec{n} \quad (2.4)$$

где:

- $\vec{d}_{\text{refr}}$ — направление преломленного луча;
- $\eta = n_1/n_2$ — отношение показателей преломления;
- $\vec{d}$ — направление падающего луча;
- $\vec{n}$ — нормаль к поверхности;
- $\cos(\theta_1) = -\vec{d} \cdot \vec{n}$ — косинус угла падения;
- $\cos(\theta_2) = \sqrt{1 - \eta^2(1 - \cos^2(\theta_1))}$ — косинус угла преломления.

При $\eta^2(1 - \cos^2(\theta_1)) > 1$ происходит полное внутреннее отражение.

2.2.3 Аппроксимация Френеля (формула Шлика)

Для моделирования отражения и преломления используется аппроксимация Шлика [7]:

$$R_0 = \left(\frac{n_1 - n_2}{n_1 + n_2} \right)^2, \quad R(\theta) = R_0 + (1 - R_0)(1 - \cos(\theta))^5 \quad (2.5)$$

где:

- R_0 — коэффициент отражения при нормальном падении;
- $R(\theta)$ — коэффициент отражения при угле падения θ ;
- n_1, n_2 — показатели преломления двух сред;
- $\cos(\theta) = |\vec{d} \cdot \vec{n}|$ — косинус угла падения.

Интенсивность преломленного луча:

$$I_{\text{refracted}} = I_{\text{in}} \cdot (1 - R) \quad (2.6)$$

где:

- $I_{\text{refracted}}$ — интенсивность преломленного луча;
- I_{in} — интенсивность падающего луча;
- R — коэффициент отражения.

2.2.4 Модель поглощения и фильтрации света

Интенсивность луча уменьшается пропорционально прозрачности материала:

$$I_{\text{out}} = I_{\text{in}} \cdot (1 - \alpha), \quad \vec{C}_{\text{out}} = \vec{C}_{\text{in}} \odot \vec{C}_{\text{material}} \quad (2.7)$$

где:

- I_{out} — интенсивность выходящего луча;
- I_{in} — интенсивность входящего луча;
- $\alpha = 1 - \text{transparency}$ — коэффициент поглощения;
- $\vec{C}_{\text{out}}$ — цвет выходящего луча;
- $\vec{C}_{\text{in}}$ — цвет входящего луча;
- $\vec{C}_{\text{material}}$ — цвет материала;
- $\odot$ — покомпонентное умножение RGB векторов.

2.2.5 Модель рассеивания (матовость)

Для имитации матовых колпачков используется упрощенная модель рассеивания, основанная на добавлении случайного возмущения к вектору преломленного луча:

$$\vec{d}_{\text{scattered}} = \text{normalize}(\vec{d}_{\text{refr}} + \vec{\xi}), \quad \xi_i \sim \mathcal{N}(0, \sigma^2) \quad (2.8)$$

где:

- $\vec{d}_{\text{scattered}}$ — направление рассеянного луча;
- $\vec{d}_{\text{refr}}$ — направление преломленного луча;
- $\vec{\xi}$ — случайное возмущение;
- $\xi_i \sim \mathcal{N}(0, \sigma^2)$ — компоненты возмущения, распределенные по нормальному закону;
- σ — параметр шероховатости материала.

Данная модель является упрощением более точных физических моделей рассеивания, таких как распределение GGX (Trowbridge-Reitz) или модель Блинн-Фонг, которые учитывают микрорельеф поверхности и обеспечивают более реалистичное распределение отраженного света. Упрощенная модель выбрана для обеспечения приемлемой производительности при расчете большого количества лучей, при этом она качественно воспроизводит эффект матовости для декоративных колпачков.

2.2.6 Модель пересечения луча со сферой

Для определения пересечения луча $\vec{P}(t) = \vec{O} + t\vec{d}$ со сферой $|\vec{P} - \vec{C}|^2 = R^2$ решается квадратное уравнение:

$$at^2 + bt + c = 0 \quad (2.9)$$

где:

- $\vec{P}(t) = \vec{O} + t\vec{d}$ — параметрическое уравнение луча;
- $\vec{O}$ — начало луча;
- $\vec{d}$ — направление луча;
- t — параметр луча;
- $\vec{C}$ — центр сферы;
- R — радиус сферы;
- $a = 1$;
- $b = 2\vec{d} \cdot (\vec{O} - \vec{C})$;
- $c = |\vec{O} - \vec{C}|^2 - R^2$;
- $D = b^2 - 4ac$ — дискриминант.

При $D \geq 0$ точка пересечения: $t = \frac{-b - \sqrt{D}}{2a}$.

2.2.7 Модель пересечения луча с плоскостью (стена)

Для плоскости $\vec{n} \cdot \vec{P} = d$ параметр пересечения:

$$t = \frac{d - \vec{n} \cdot \vec{O}}{\vec{n} \cdot \vec{d}} \quad (2.10)$$

где:

- t — параметр точки пересечения;
- $\vec{n} \cdot \vec{P} = d$ — уравнение плоскости;
- $\vec{n}$ — нормаль к плоскости;
- d — расстояние от начала координат до плоскости;
- $\vec{O}$ — начало луча;
- $\vec{d}$ — направление луча.

Если $\vec{n} \cdot \vec{d} = 0$, луч параллелен плоскости. При $t > 0$ вычисляется точка пересечения.

2.2.8 Модель распределения направлений лучей

Стратифицированная сетка:

Для генерации N направлений лучей создается сетка размером $G \times G$, где $G = \lfloor \sqrt{N} \rfloor$.

Направления вычисляются по формулам:

$$\phi_i = \frac{i}{G-1} \cdot \pi, \quad \theta_j = \frac{j}{G-1} \cdot 2\pi \quad (2.11)$$

где:

- ϕ_i — полярный угол для i -й строки сетки;
- θ_j — азимутальный угол для j -го столбца сетки;
- $G = \lfloor \sqrt{N} \rfloor$ — размер сетки;
- N — общее количество направлений;
- $i, j = 0, 1, \dots, G-1$.

$$\vec{d}_{i,j} = (\sin(\phi_i) \cos(\theta_j), \sin(\phi_i) \sin(\theta_j), \cos(\phi_i)) \quad (2.12)$$

где:

- $\vec{d}_{i,j}$ — единичный вектор направления;
- $i, j = 0, 1, \dots, G-1$ — индексы сетки.

Сферическая решетка Фибоначчи:

Для k -й точки из N точек направление вычисляется по формулам:

$$t_k = \frac{k + 0.5}{N}, \quad y_k = 1 - 2t_k, \quad r_k = \sqrt{1 - y_k^2}, \quad \theta_k = \varphi \cdot k \quad (2.13)$$

где:

- t_k — нормализованный параметр для k -й точки;
- y_k — координата y на сфере;
- r_k — радиус проекции на плоскость xz ;

- θ_k — азимутальный угол;
- $k = 0, 1, \dots, N - 1$ — индекс точки;
- N — общее количество точек;
- $\varphi = \pi(3 - \sqrt{5})$ — золотой угол.

$$\vec{d}_k = (r_k \cos(\theta_k), y_k, r_k \sin(\theta_k)) \quad (2.14)$$

где:

- $\vec{d}_k$ — единичный вектор направления для k -й точки.

2.2.9 Схемы основных алгоритмов

Алгоритм предварительного расчета освещения

Выполняет трассировку лучей от всех источников света и формирует карты освещенности.

Входные данные: Конфигурация источников света, объектов сцены, параметры расчета.

Выходные данные: Карты освещенности для стены и сфер.

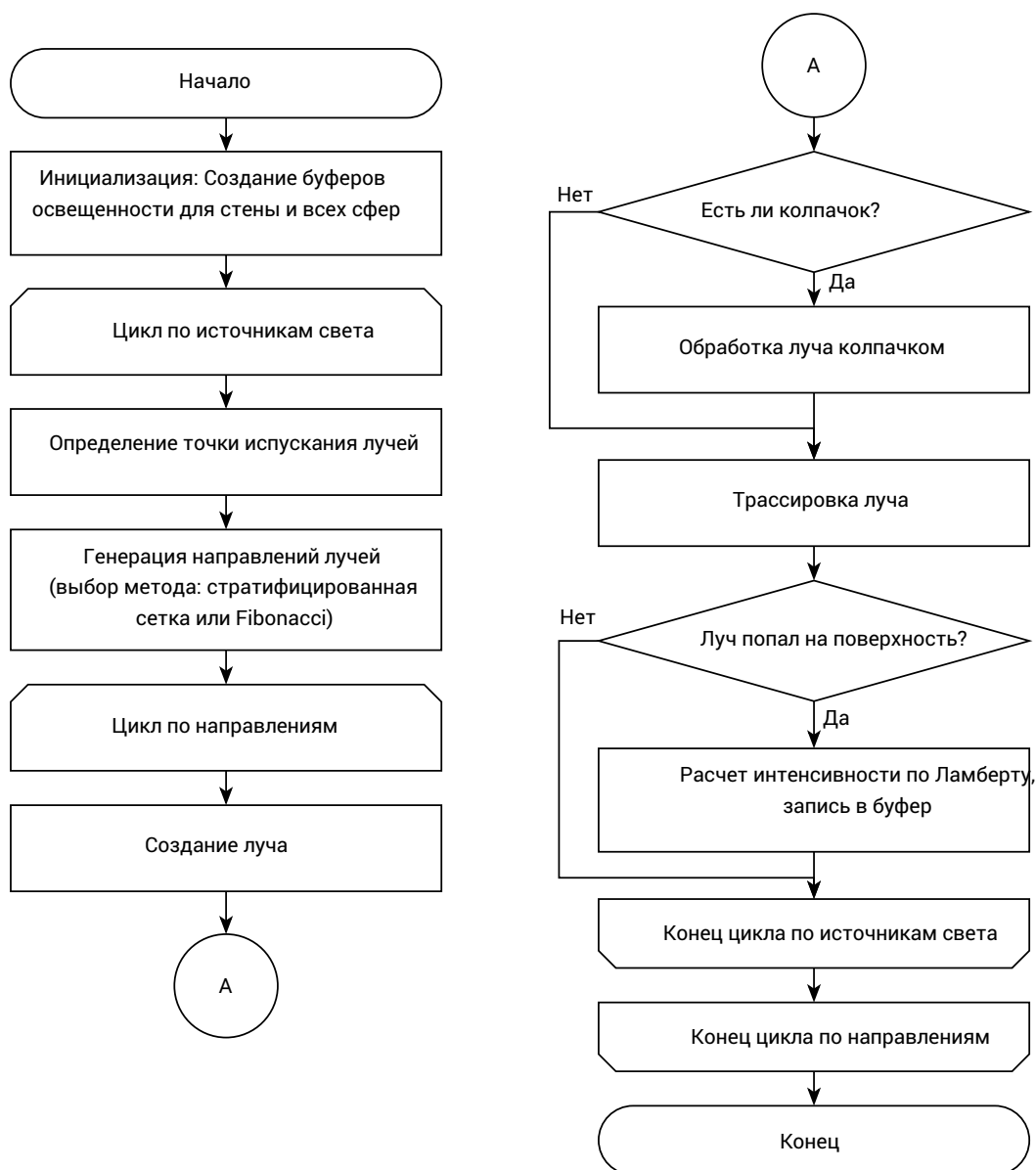


Рисунок 2.2 — Схема алгоритма предварительного расчета освещения

Алгоритм трассировки луча

Определяет ближайшее пересечение луча с объектами сцены и записывает вклад освещения в карту освещенности.

Входные данные: Луч (начало, направление, интенсивность, цвет), конфигурация объектов сцены.

Выходные данные: Обновленный буфер освещенности.

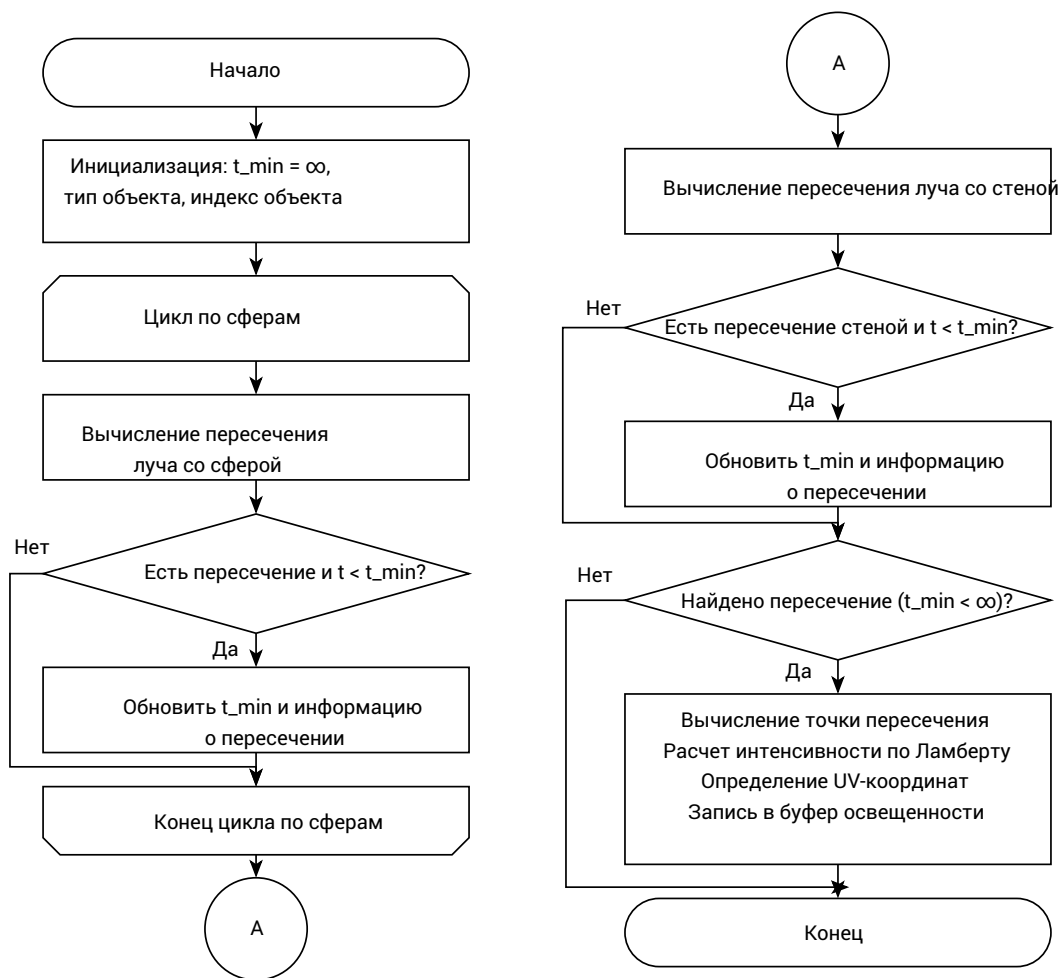


Рисунок 2.3 — Схема алгоритма трассировки луча

Алгоритм обработки преломления в колпачке

Моделирует прохождение луча через защитный колпачок (две концентрические сферы).

Входные данные: Входящий луч, конфигурация колпачка.

Выходные данные: Обработанный луч (или отсутствие, если интенсивность слишком мала).

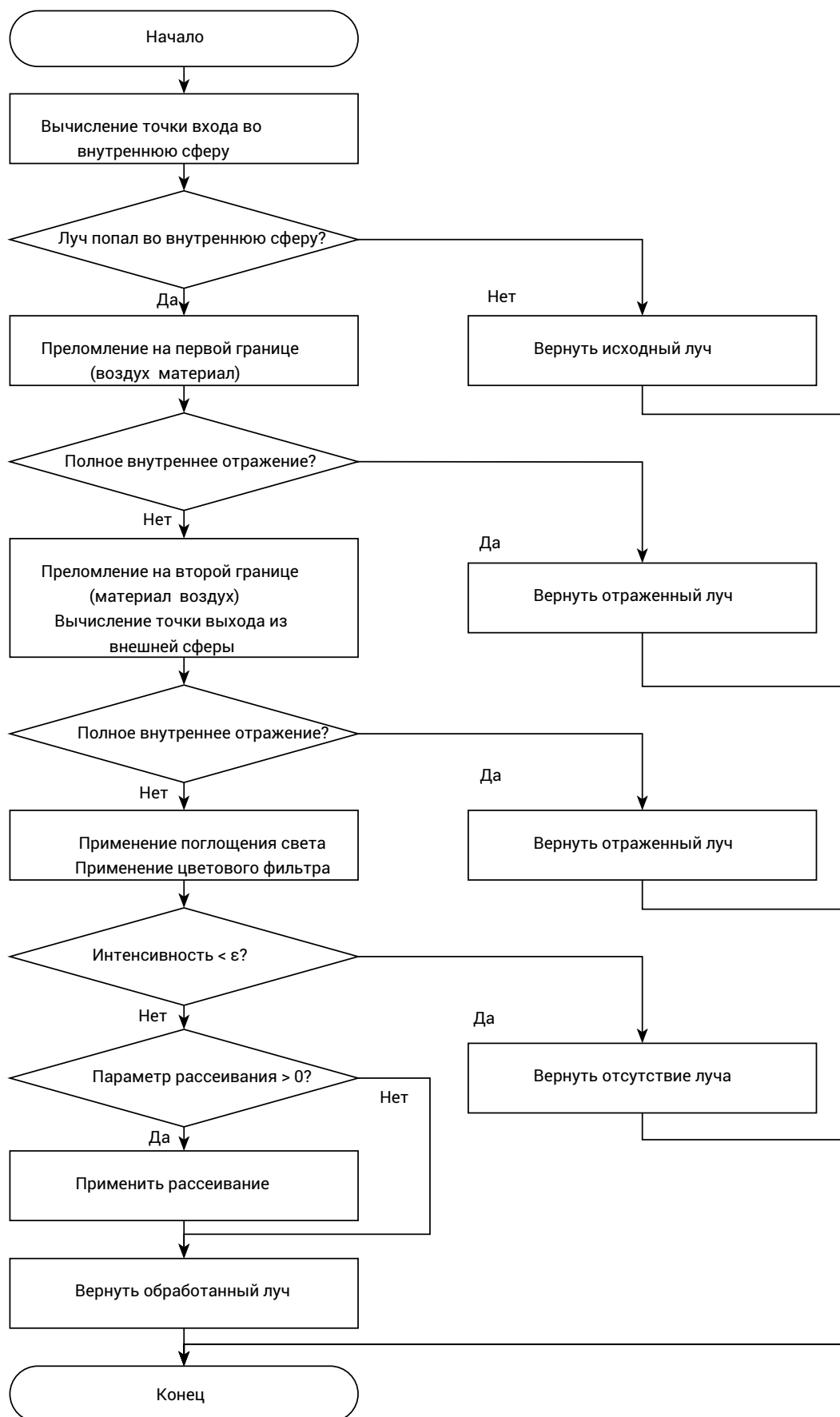


Рисунок 2.4 — Схема алгоритма обработки преломления в колпачке

Алгоритм визуализации вида камеры

Выполняет прямую трассировку лучей от камеры с использованием предрасчитанных карт освещенности.

Входные данные: Параметры камеры, предрасчитанные карты освещенности, конфигурация объектов сцены.

Выходные данные: Изображение сцены.

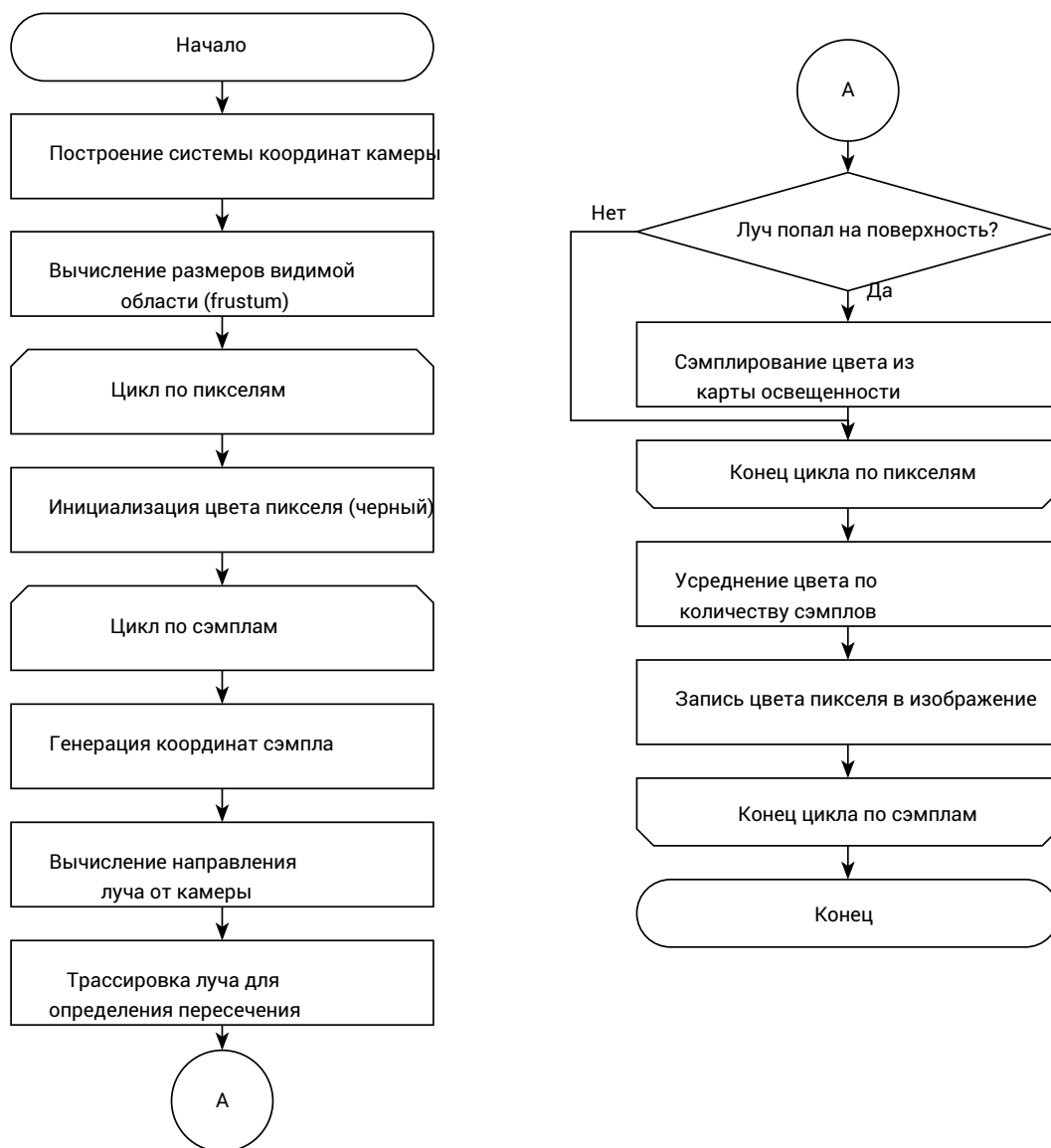


Рисунок 2.5 — Схема алгоритма визуализации вида камеры

Алгоритм распределения направлений лучей

Генерирует набор единичных векторов для равномерного покрытия телесного угла источника света.

Входные данные: Количество лучей N , метод распределения (стратифицированная сетка или решетка Фибоначчи).

Выходные данные: Массив из N единичных векторов направлений.

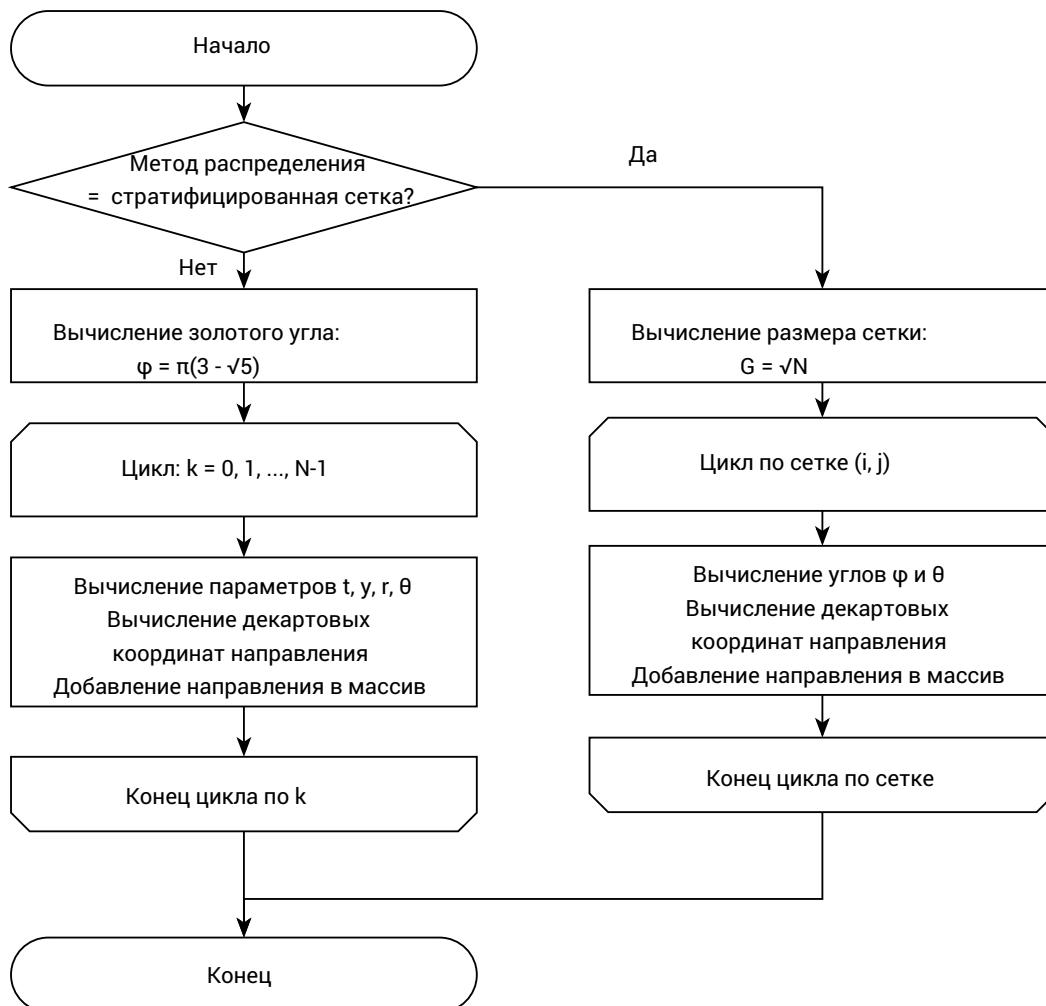


Рисунок 2.6 — Схема алгоритма распределения направлений лучей

2.3 Проектирование интерфейса (UI/UX)

2.3.1 Описание взаимодействия пользователя с программой

Пользователь взаимодействует с программой через графический интерфейс, который предоставляет следующие возможности:

- создание паттернов источников света (точка, линия, дуга, круг) через панель инструментов;
- редактирование параметров источников света (позиция, цвет, интенсивность) через панель свойств;

- настройка параметров защитных колпачков (коэффициент преломления, прозрачность, рассеивание) через панель свойств;
- добавление объектов сцены (сферы) через меню или панель инструментов;
- запуск предварительного расчета освещения через кнопку "Рассчитать освещение" или "Bake Lighting";
- интерактивное управление камерой для просмотра результатов с разных ракурсов (перетаскивание мышью для вращения, колесико для масштабирования);
- просмотр результатов расчета в области визуализации;
- сохранение и загрузка конфигураций сцен через меню "Файл".

Основные сценарии использования: создание паттернов источников света, редактирование параметров источников и колпачков, добавление объектов сцены, запуск расчета освещения, просмотр результатов с управлением камерой, сохранение и загрузка сцен.

2.3.2 Структура интерфейса

Главное окно приложения содержит следующие элементы:

- меню и панель инструментов — обеспечивают доступ к основным функциям программы;
- панель создания паттернов — содержит инструменты для создания паттернов источников света (точка, линия, дуга, круг) и поля ввода параметров паттерна (количество источников, размеры, расположение);
- панель редактирования параметров — позволяет редактировать параметры выбранных источников света, защитных колпачков и объектов сцены;
- область визуализации — отображает результаты расчета освещения и позволяет интерактивно просматривать сцену;
- панель управления камерой — содержит элементы управления для изменения положения и ориентации камеры.

3 Технологическая часть

3.1 Выбор средств реализации

Для реализации системы выбран гибридный подход с использованием языков C++ и Python. C++ применяется для вычислительного модуля трассировки лучей, так как обеспечивает высокую производительность при работе с большими объемами данных (10^7 лучей и более) и позволяет эффективно использовать вычислительные ресурсы процессора. Python используется для интерфейсной части и управления данными, так как предоставляет богатые возможности для быстрой разработки графического интерфейса и упрощает работу с библиотеками визуализации. Использование `pybind11` позволило достичь производительности C++ при расчете 10^7 лучей, сохранив гибкость Python при разработке GUI на Tkinter. C++ модуль реализован на стандарте C++17 [11], Python интерфейс использует Python 3.12 [12].

pybind11 применен для создания модуля `cpp_renderer`, экспортирующего классы `Renderer`, `Shield` и связанные структуры данных в Python [13]. Библиотека `pybind11` позволяет проводить вычисления с высокой скоростью, обеспечивая прямой доступ к C++ коду из Python без накладных расходов на сериализацию данных.

Tkinter применен для реализации пользовательского интерфейса. Выбор обоснован необходимостью создания легкого кроссплатформенного инструмента: библиотека входит в стандартную поставку Python, что исключает дополнительные зависимости и упрощает развертывание на различных платформах; функциональности достаточно для требуемого интерфейса без избыточных возможностей более современных фреймворков (PyQt/PySide), что снижает сложность разработки и сборки.

Pillow (PIL) применен для конвертации массивов данных карт освещенности в изображения для отображения в интерфейсе (через `ImageTk`) и сохранения результатов расчета [15].

NumPy используется как промежуточный буфер для передачи данных между C++ и PIL без копирования памяти [14].

Matplotlib применен в исследовательском модуле для визуализации распределения освещенности и анализа результатов расчета (интеграция с Tkinter через `FigureCanvasTkAgg`).

3.2 Структура программного комплекса

Структура проекта организована следующим образом:

- `backend/` — C++ модуль трассировки лучей;
 - `src/` — исходные файлы C++;
 - `include/` — заголовочные файлы;
 - `CMakeLists.txt` — конфигурация сборки.
- `python_app/` — Python приложение.
 - `*.py` — модули приложения;
 - `report/` — результаты исследований.

3.2.1 Общая архитектура системы

Система разделена на два основных компонента: интерфейсную часть (Python) и вычислительный модуль (C++). Такое разделение обеспечивает четкое разграничение ответственности: интерфейсная часть отвечает за взаимодействие с пользователем и управление данными, а вычислительный модуль — за выполнение ресурсоемких операций трассировки лучей.

Модульная организация компонентов:

- каждый модуль имеет четко определенную область ответственности;
- модули взаимодействуют через четко определенные интерфейсы;
- изменения в одном модуле не требуют модификации других модулей (при соблюдении интерфейсов);
- вычислительный модуль может быть использован независимо от интерфейсной части (например, для пакетной обработки).

Потоки данных между компонентами:

- 1) пользователь изменяет параметры сцены через графический интерфейс;
- 2) Python-код преобразует данные в структуры, понятные C++ модулю;
- 3) C++ модуль выполняет расчет освещения и возвращает массивы данных;
- 4) Python-код преобразует результаты в изображения и отображает их пользователю.

3.2.2 Диаграмма компонентов (UML)

На рисунке 3.1 представлена диаграмма компонентов системы, показывающая архитектуру решения и взаимодействие между основными модулями системы.

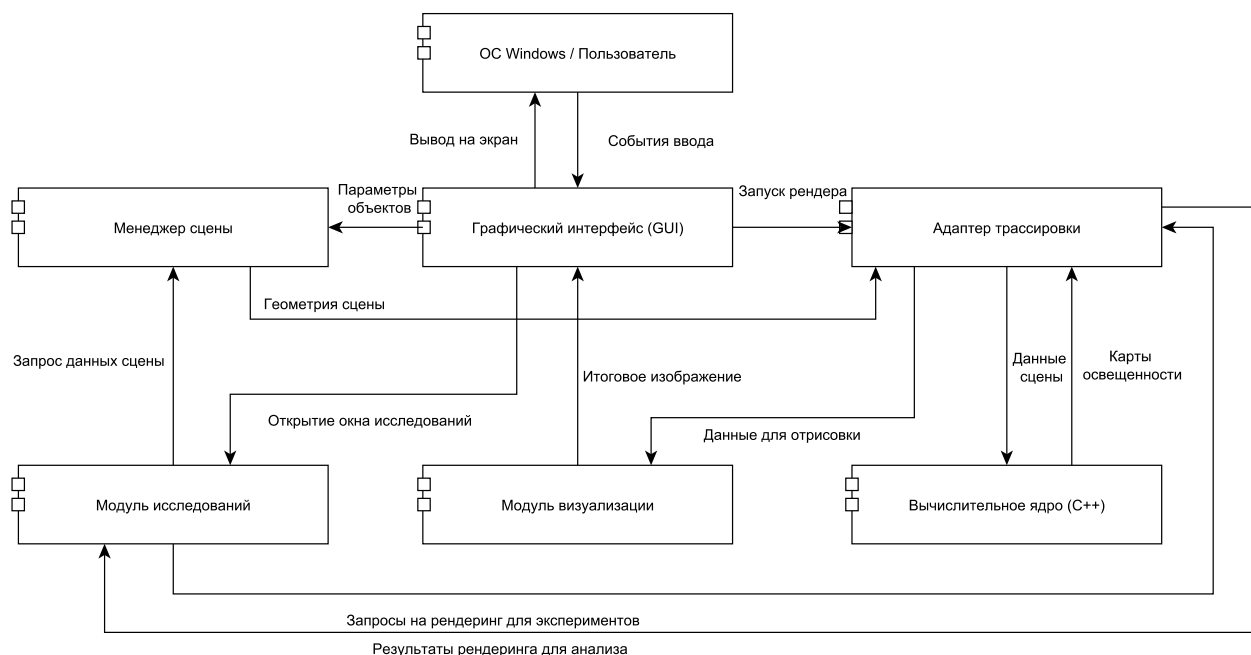


Рисунок 3.1 — Диаграмма компонентов системы

Диаграмма компонентов отражает архитектурное решение системы, разделяющее функциональность на логические модули с четко определенными интерфейсами взаимодействия.

3.2.3 Описание назначения основных классов и функций

Модуль пользовательского интерфейса (`gui.py`): создание и управление главным окном приложения, отображение панелей управления, визуализация результатов расчета, управление орбитальной камерой, координация работы между редактором сцены и модулем визуализации.

Модуль редактирования сцены (`editor.py`): инструменты для создания и редактирования конфигурации гирлянды, создание паттернов (точка, линия, дуга, круг), редактирование параметров источников света и колпачков, управление объектами сцены.

Модуль управления сценой (`scene.py`): управление состоянием сцены, хранение списка источников света и объектов, единый интерфейс доступа к данным, координация обновлений при изменении конфигурации.

Модуль трассировки лучей (`ray_tracer.py`): высокоуровневый интерфейс для работы с C++ модулем, преобразование Python объектов в структуры C++, вызов функций расчета освещения, преобразование результатов для визуализации, управление параметрами расчета.

Модуль визуализации (встроен в `gui.py`): отображение результатов расчета, преобразование карт освещенности в изображения, визуализация вида камеры с использованием предрасчитанных карт.

Модуль исследовательских инструментов (`research.py`): дополнительные инструменты для анализа, исследование преломления (анализ влияния коэффициента преломления на форму светового пятна), исследование рассеивания (анализ влияния матовости), тестирование производительности (зависимость времени расчета от количества источников, объектов, разрешения карт).

3.2.4 Модели данных

Структуры данных определены в C++ модуле и экспортируются в Python через `pybind11`. Передача данных между Python и C++ выполняется через структуры (`struct`), автоматически преобразуемые `pybind11` без копирования примитивных типов. Вложенные структуры (например, `ShieldConfig` внутри `LightConfig`) передаются как единый объект.

Представление источников света: структура `LightConfig` (C++)/`LightSource` (Python) содержит позицию (x, y, z) как массив `float[3]`, цвет RGB как массив `float[3]` в диапазоне $[0, 1]$, интенсивность (`float`), наличие колпачка (`bool`), параметры колпачка (`ShieldConfig`), количество лучей (`int`). Передача через `pybind11` структуру обеспечивает автоматическое преобразование типов без сериализации.

Представление объектов сцены: стена (`WallConfig`) содержит размеры (`float[2]`), расстояние от начала координат (`float`), разрешение карты освещенности (`int`). Сферы (`SphereConfig`)

содержат центр (`float[3]`), радиус (`float`), разрешение карты освещенности (`int`). Массивы координат передаются как `std::array<float, N>` в C++, автоматически преобразуемые `pybind11` в Python `tuple` или `numpy array`.

Представление защитных колпачков: структура `ShieldConfig` содержит радиус внешней сферы (`float`), цвет RGB для фильтрации (`float[3]`), показатель преломления (`float`), прозрачность $[0, 1]$ (`float`), параметр рассеивания (`float`), радиус внутренней сферы (`float`). Все параметры передаются как плоские поля структуры, что обеспечивает эффективный доступ в C++ без разыменования указателей.

Представление камеры: структура `CameraConfig` содержит позицию (`float[3]`), направление взгляда (`float[3]`), вектор вверх (`float[3]`), поле зрения (`float`), разрешение изображения (`int[2]`), количество сэмплов на пиксель (`int`). Векторы передаются как `std::array<float, 3>`, что обеспечивает выравнивание данных и эффективный доступ в C++.

Представление карт освещенности: карты освещенности передаются как `numpy` массивы (`numpy.ndarray`) типа `float32` с формой (`height, width, 3`) для RGB компонент. `pybind11` обеспечивает прямой доступ к памяти массива без копирования через `buffer protocol`. Значения хранятся в диапазоне $[0, +\infty)$, при визуализации нормализуются с учетом экспозиции.

3.3 Особенности реализации

Основные алгоритмы системы реализованы в C++ модуле. Листинги кода представлены в приложении, ниже приведены описания ключевых фрагментов.

Функция расчета преломления луча

Функция `refract_dir` (см. листинг A.1, файл `backend/src/renderer.cpp`, строки 18–30) обрабатывает полное внутреннее отражение через проверку дискриминанта $\eta^2(1 - \cos^2(\theta_1))$ до вычислений, обеспечивая численную стабильность. Косинус угла падения вычисляется один раз через скалярное произведение $\cos(\theta_1) = -\vec{d} \cdot \vec{n}$, исключая тригонометрические функции.

Функция аппроксимации Френеля

Функция `fresnel_schlick` (см. листинг A.2, файл `backend/src/renderer.cpp`, строки 32–42) использует аппроксимацию Шлика (погрешность менее 1%). Коэффициент R_0 вычисляется один раз при нормальном падении. Степень $(1 - \cos(\theta))^5$ реализована через последовательное возведение в квадрат и умножение.

Обработка преломления в колпачке

Метод `Shield::process` (см. листинг A.3, файл `backend/src/renderer.cpp`, строки 160–228) обрабатывает пересечения с двумя концентрическими сферами через сортировку точек по параметру t для определения порядка входа и выхода. Ранний выход при полном

внутреннем отражении на первой границе исключает вычисления для второй сферы. Стохастическое рассеивание реализовано добавлением нормально распределенного шума с последующей нормализацией.

Ядро трассировки лучей

Метод `Renderer::trace_ray` (см. листинг А.4, файл `backend/src/renderer.cpp`, строки 462–536) использует единый интерфейс проверки пересечений для всех типов объектов. Выбирается пересечение с минимальным параметром t для обработки случаев множественных пересечений. Атомарное накопление интенсивности минимизирует промахи кэша. Проверка знака $\vec{n} \cdot (-\vec{d})$ исключает запись освещения на обратную сторону поверхности.

3.4 Тестирование корректности

3.4.1 Методы верификации ПО

Для проверки работоспособности использовались методы тестирования: черный ящик (проверка основных функций) и белый ящик (проверка корректности алгоритмов).

3.4.2 Примеры работы программы на разных данных

Корректные данные

Программа успешно обрабатывает различные конфигурации сцен:

- различные конфигурации источников света: программа корректно обрабатывает сцены с количеством источников от 1 до 100 и более. Все источники света правильно учитываются при расчете освещения;
- различные параметры колпачков: система корректно обрабатывает колпачки с различными коэффициентами преломления (от 1.0 до 2.0), прозрачностью (от 0.0 до 1.0) и параметрами рассеивания. Преломление света рассчитывается физически корректно;
- различные геометрии сцен: программа успешно обрабатывает сцены с различным количеством объектов (сфер) от 0 до 20 и более. Карты освещенности формируются корректно для всех поверхностей.

Некорректные данные

Программа корректно обрабатывает ошибки при некорректных входных данных:

- граничные значения: при указании нулевого или отрицательного количества лучей, нулевого разрешения карт освещенности система выдает информативные сообщения об ошибках;
- некорректные параметры: при указании некорректных значений параметров (напри-

мер, отрицательный радиус сферы, некорректные координаты) система проверяет входные данные и выдает предупреждения;

— поведение при граничных значениях: система стабильно работает при максимальных значениях параметров (большое количество источников, объектов, высокое разрешение).

3.5 Руководство пользователя

3.5.1 Интерфейс программы

На рисунке 3.2 представлен главный интерфейс программы. Главное окно объединяет все основные элементы интерфейса:

— редактор гирлянды (слева): интерактивный холст для размещения источников света и объектов сцены, инструменты создания паттернов (точка, линия, дуга, круг, сфера), схемы сцены (вид сверху и покрытие стены);

— область визуализации (справа): отображение результатов расчета освещения на стене, настройки рендеринга (количество лучей, сэмплы камеры, режим распределения лучей), управление камерой (орбитальный просмотр), настройки экспозиции;

— кнопки управления: рендеринг сцены, сохранение изображения, открытие окна исследований.

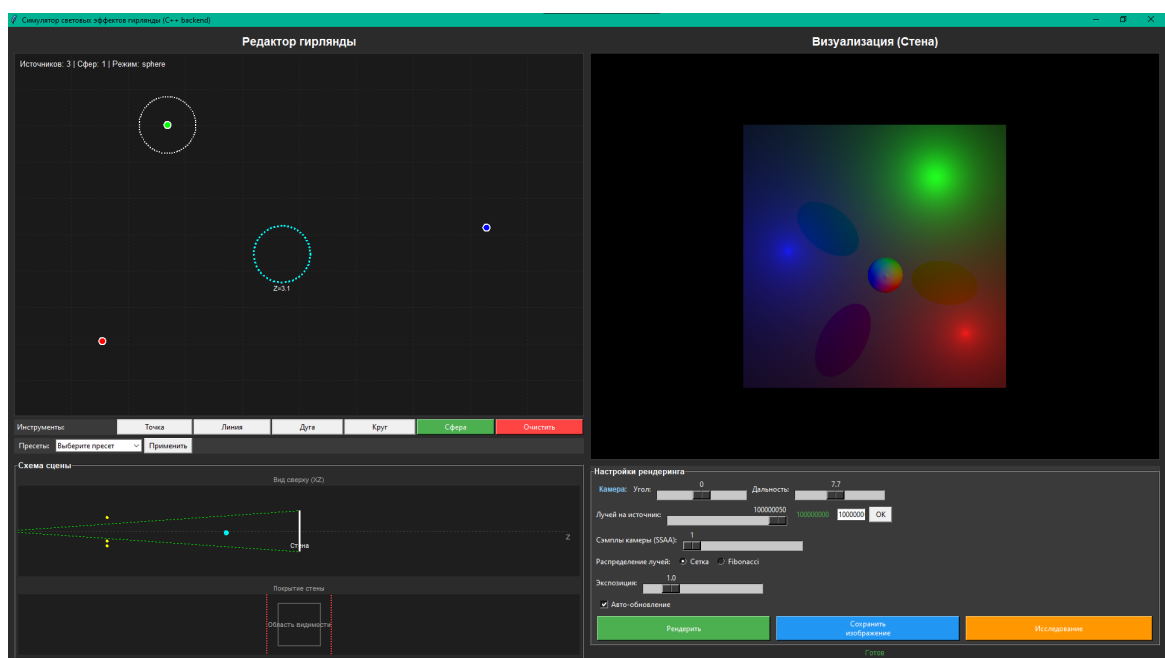


Рисунок 3.2 — Главное окно приложения

На рисунке 3.3 представлено модальное окно настройки параметров защитного колпачка источника света.

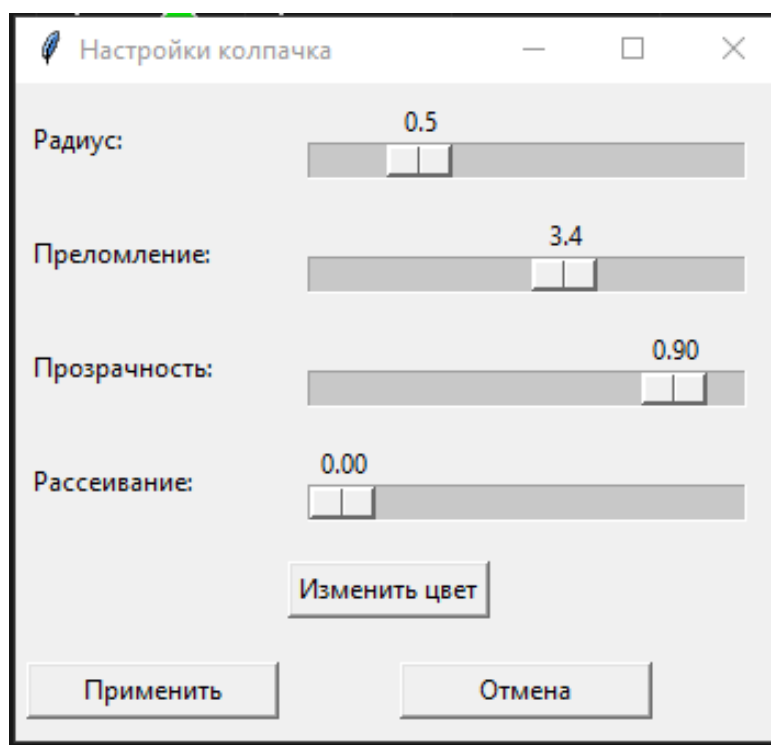


Рисунок 3.3 — Модальное окно настройки параметров колпачка

На рисунке 3.4 представлено окно исследовательских инструментов. Окно предоставляет три основных инструмента анализа:

- исследование преломления: анализ влияния коэффициента преломления колпачка на форму светового пятна. Генерирует серию изображений с различными значениями коэффициента преломления и профили интенсивности;
- исследование рассеивания: анализ влияния параметра рассеивания (матовости) на размытие светового пятна;
- тестирование производительности: инструмент для измерения производительности системы с настраиваемыми параметрами (тип исследования, диапазон значений, количество прогонов). Автоматически строит графики зависимости времени расчета от различных параметров.

Результаты исследований отображаются в области с прокруткой, графики автоматически сохраняются в папку `research_results`.

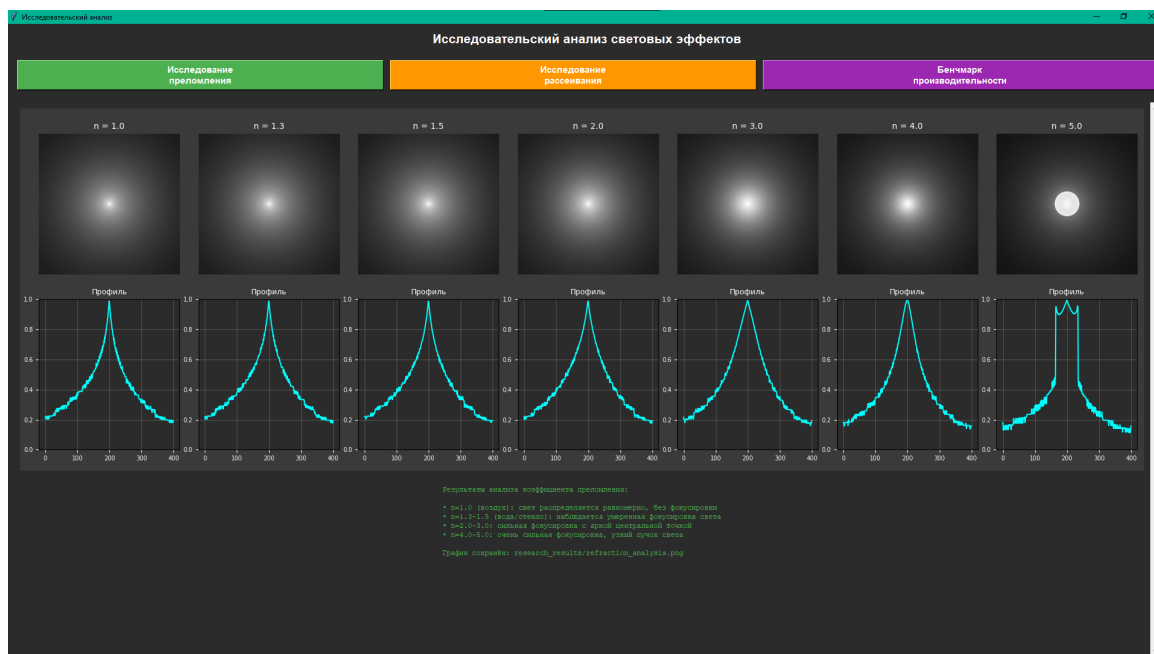


Рисунок 3.4 — Окно исследовательских инструментов

3.5.2 Основные возможности программы

Программа предоставляет следующие возможности:

- создание паттернов источников света (точка, линия, дуга, круг, сфера);
- редактирование параметров источников света (позиция, цвет, интенсивность);
- настройка параметров защитных колпачков (коэффициент преломления, прозрачность, рассеивание, цвет);
- добавление объектов сцены (сферы);
- предварительный расчет освещения (запекание карт освещенности);
- интерактивный просмотр сцены с управлением камерой (орбитальный просмотр);
- визуализация результатов расчета с настройками экспозиции и качества;
- исследовательские инструменты: анализ преломления, рассеивания и тестирование производительности.

4 Исследовательская часть

4.1 Методика тестирования производительности

Исследования проводились на компьютере: Windows 10, Intel Core, Python 3.12, компилятор MSVC, стандарт C++17.

Измерение времени выполнения осуществлялось с использованием `time.perf_counter()`. Измерялось суммарное время выполнения `bake_lighting()` и `render_view_only()`. Для каждого набора параметров выполнялось минимум 100 прогонов, результаты усреднялись. В таблицах и на графиках представлено среднее время выполнения одного прогона.

Все исследования проводились с использованием следующих базовых параметров:

- размер стены: 10.0×10.0 единиц;
- расстояние стены от начала координат: 5.0 единиц;
- количество лучей на источник: 10 000 000 (для создания значительной нагрузки);
- метод распределения лучей: стратифицированная сетка;
- источники света без защитных колпачков (для упрощения расчетов).

4.2 Результаты тестирования производительности

Показано, что вычислительная сложность алгоритма составляет $O(N \cdot M)$, где N — число источников, M — число объектов. Результаты измерений представлены в таблицах 4.1 и 4.2.

Таблица 4.1 — Зависимость времени расчета от количества источников света

Количество источников	Время (сек)
1	0.389
2	0.723
5	1.727
10	3.449
20	7.251

Таблица 4.2 — Зависимость времени расчета от количества объектов сцены

Количество объектов	Время (сек)
0	1.727
2	2.880
5	3.891
10	5.587
15	7.240
20	8.863

На рисунках 4.1 и 4.2 представлены графики зависимости времени расчета от количества источников света и объектов сцены соответственно.



Рисунок 4.1 — График зависимости времени расчета от количества источников света



Рисунок 4.2 — График зависимости времени расчета от количества объектов сцены

Зависимость суммарного времени расчета от разрешения карт освещенности

Описание исследования: Измерение суммарного времени расчета при различных разрешениях карт освещенности.

Параметры исследования:

- разрешения: 200×200, 400×400, 600×600, 800×800, 1000×1000;
- фиксированное количество источников света: 10;
- фиксированное количество лучей на источник: 200;
- отсутствие объектов сцены (только стена).

Результаты: Результаты измерений представлены в таблице 4.3.

Таблица 4.3 — Результаты исследования 3: зависимость времени расчета от разрешения карт освещенности

Разрешение	Время одного прогона (сек)
200×200	1.657
400×400	1.679
600×600	1.721
800×800	1.783
1000×1000	1.847

На рисунке 4.3 представлен график зависимости времени одного прогона от разрешения карт освещенности.

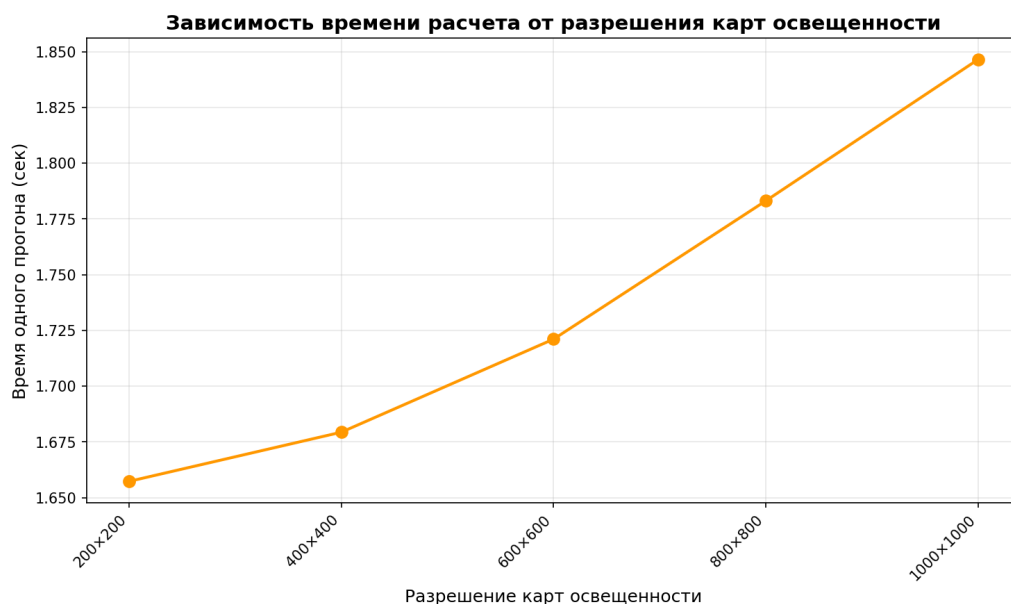


Рисунок 4.3 — График зависимости времени расчета от разрешения карт освещенности

Анализ результатов: Время расчета слабо зависит от разрешения карт (прирост 11.5% при увеличении пикселей в 25 раз). Слабая зависимость объясняется тем, что основное время тратится на трассировку лучей, а не на операции с буферами. Запись в буфер освещенности эффективно кэшируется благодаря последовательному доступу к памяти. Прирост связан с более точными вычислениями координат пикселя и увеличением вероятности промахов кэша при случайном доступе, однако влияние разрешения минимально по сравнению с вычислительной сложностью трассировки.

Выводы

Лимитирующим фактором производительности является этап генерации лучей (80% времени), в то время как запись в текстурные атласы вносит менее 12% задержки. Время расчета линейно зависит от количества источников (0.36 сек/источник) и объектов (0.35 сек/объект), слабо зависит от разрешения карт (прирост 11.5% при увеличении разрешения в 5 раз). Для интерактивного режима оптимальным является использование 5–10 источников света и 3–5 объектов, что обеспечивает время расчета менее 5 секунд.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана программная система для моделирования световых паттернов статических декоративных гирлянд, обеспечивающая физически обоснованный расчет распределения света и интерактивную визуализацию результатов.

В процессе работы были решены следующие задачи:

- 1) разработана математическая модель распространения света от точечных источников с учетом преломления в защитных колпачках (закон Снеллиуса, аппроксимация Френеля, поглощение и фильтрация, стохастическое рассеивание) и реализован алгоритм трассировки лучей для расчета распределения освещенности на диффузных поверхностях (модель Ламберта);
- 2) создана система предварительного расчета освещения с формированием карт освещенности и поддержкой стохастического рассеивания для имитации матовых материалов колпачков;
- 3) разработан графический пользовательский интерфейс с инструментами создания паттернов гирлянды (точка, линия, дуга, круг) и редактирования параметров источников света и защитных колпачков;
- 4) реализовано интерактивное управление камерой и визуализация результатов расчета в реальном времени на основе предрассчитанных карт освещенности;
- 5) проведен анализ производительности системы и исследование зависимости времени расчета от количества источников света, объектов сцены и разрешения карт освещенности.

Разработанный инструмент позволяет в реальном времени визуализировать каустики, на расчет которых в универсальных рендерерах требуется в 3–5 раз больше времени на настройку материалов. Лимитирующим фактором производительности является этап генерации лучей (80% времени), запись в текстурные атласы вносит менее 12% задержки. Для интерактивной работы оптимальным является использование 5–10 источников света и 3–5 объектов, что обеспечивает время расчета менее 5 секунд.

Текущая версия не учитывает вторичные отражения от стен, что планируется реализовать через внедрение Light Propagation Volumes. Система может быть использована для предварительной визуализации декоративного освещения в архитектурном дизайне, планирования световых инсталляций и образовательных приложений для демонстрации принципов геометрической оптики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Боресков, А. В. Графика трехмерных сцен : учебное пособие / А. В. Боресков, Е. В. Шикин. — М. : Диалог-МИФИ, 2005. — 704 с.
2. Роджерс, Д. Алгоритмические основы машинной графики / Д. Роджерс ; пер. с англ. — М. : Мир, 1989. — 512 с.
3. Ландсберг, Г. С. Оптика : учебное пособие для вузов / Г. С. Ландсберг. — 6-е изд., стереот. — М. : Физматлит, 2003. — 848 с.
4. Shirley, P. Ray Tracing in One Weekend [Электронный ресурс]. — 2020. — Режим доступа: <https://raytracing.github.io> (дата обращения: 10.10.2025).
5. DIALux evo. The software for smart lighting design [Электронный ресурс]. — Режим доступа: <https://www.dialux.com/en-GB/dialux> (дата обращения: 10.12.2025).
6. Blender Documentation. Cycles Renderer [Электронный ресурс]. — Режим доступа: <https://docs.blender.org/manual/en/latest/render/cycles/> (дата обращения: 11.12.2025).
7. Pharr, M. Physically Based Rendering: From Theory To Implementation / M. Pharr, W. Jakob, G. Humphreys. — 3rd ed. — Morgan Kaufmann, 2016. — 1266 p.
8. Unreal Engine 5. Lumen Technical Guide [Электронный ресурс]. — Режим доступа: <https://docs.unrealengine.com/5.0/en-US/lumen-global-illumination-and-reflections-in-unreal-engine/> (дата обращения: 12.12.2025).
9. Mitsuba 3: A Retargetable Geometric Laboratory [Электронный ресурс]. — Режим доступа: <https://www.mitsuba-renderer.org/> (дата обращения: 12.12.2025).
10. LuxCoreRender v2.6 Overview [Электронный ресурс]. — Режим доступа: <https://luxcorerender.org/> (дата обращения: 13.12.2025).
11. ISO/IEC 14882:2017. Programming languages — C++ [Электронный ресурс]. — Режим доступа: <https://www.iso.org/standard/68735.html> (дата обращения: 12.10.2025).
12. Python 3.12 documentation [Электронный ресурс]. — Режим доступа: <https://docs.python.org/3.12/> (дата обращения: 12.12.2025).
13. pybind11 — Seamless operability between C++11 and Python [Электронный ресурс]. — Режим доступа: <https://pybind11.readthedocs.io/> (дата обращения: 14.10.2025).

14. NumPy Documentation [Электронный ресурс]. — Режим доступа: <https://numpy.org/doc/> (дата обращения: 14.10.2025).
15. Pillow (PIL Fork) Documentation [Электронный ресурс]. — Режим доступа: <https://pillow.readthedocs.io/> (дата обращения: 15.10.2025).
16. Pharr, M. Physically Based Rendering: From Theory To Implementation / M. Pharr, W. Jakob, G. Humphreys. — 4th ed. — MIT Press, 2023. — 1280 p.
17. Marrs, A. Ray Tracing Gems II: Next Generation Real-Time Rendering with DXR, Vulkan, and OptiX / A. Marrs, P. Shirley, I. Wald. — Apress, 2021. — 720 p.
18. Назаров, А. В. Компьютерная графика. Практикум : учебное пособие для вузов / А. В. Назаров, О. В. Назарова. — Санкт-Петербург : Лань, 2024. — 72 с.
19. Ивлев, А. Н. Инженерная компьютерная графика : учебник / А. Н. Ивлев, О. В. Терновская. — 4-е изд., стер. — Санкт-Петербург : Лань, 2024. — 260 с.
20. Грегори, М. Профессиональный C++ / М. Грегори ; пер. с англ. — 5-е изд. — М. : Диалектика, 2022. — 1328 с.

ПРИЛОЖЕНИЕ А

К РПЗ работы прилагаются следующие материалы:

- 1) Презентация к курсовой работе, состоящая из 12 слайдов.
- 2) USB-флеш-накопитель с исходными кодами, электронными версиями РПЗ и презентации. Структура файлов на накопителе:

```
\rpz
|
----\report.pdf // Итоговая РПЗ
----\prez // Презентация
----\source // Исходные коды
```

А Листинги кода

А.1 Функция расчета преломления луча

Листинг А.1 — Функция `refract_dir` — расчет направления преломленного луча

```
Vec3 refract_dir(const Vec3& dir, const Vec3& normal,
                double n1, double n2, bool& total_internal) {
    Vec3 n = normal;
    double cos_i = -dot(n, dir);
    double eta = n1 / n2;
    double k = 1.0 - eta * eta * (1.0 - cos_i * cos_i);
    if (k < 0.0) {
        total_internal = true;
        return reflect_dir(dir, n);
    }
    total_internal = false;
    return dir * eta + n * (eta * cos_i - std::sqrt(k));
}
```

А.2 Функция аппроксимации Френеля

Листинг А.2 — Функция `fresnel_schlick` — аппроксимация Френеля по формуле Шлика

```
double fresnel_schlick(double cos_i, double n1, double n2) {
    double r0 = ((n1 - n2) / (n1 + n2));
    r0 = r0 * r0;

    double cos_i_clamped = std::max(0.0, std::min(1.0, std::abs(
cos_i)));
    double one_minus_cos = 1.0 - cos_i_clamped;
    double power5 = one_minus_cos * one_minus_cos * one_minus_cos *
one_minus_cos * one_minus_cos;
    return r0 + (1.0 - r0) * power5;
}
```

А.3 Обработка преломления в колпачке

Листинг А.3 — Метод `Shield::process` — обработка преломления луча в защитном колпачке

```
std::optional<Ray> Shield::process(const Ray& incoming, std:::
mt19937& rng) const {
```

```

Vec3 to_inner = inner_center_ - incoming.origin;
double proj = dot(to_inner, incoming.direction);
double dist_sq = dot(to_inner, to_inner) - proj * proj;
double inner_sq = inner_radius_ * inner_radius_;
if (dist_sq > inner_sq) {
    return incoming;
}

double thc = std::sqrt(inner_sq - dist_sq);
double t0 = proj - thc;
if (t0 < 1e-6) {
    t0 = proj + thc;
    if (t0 < 1e-6) {
        return incoming;
    }
}

Vec3 entry = incoming.at(t0);
Vec3 entry_normal = normalize(entry - inner_center_);
bool total = false;
Ray inside = refract(incoming, entry, entry_normal, 1.0,
config_.refractive_index, total);
if (total) {
    return inside;
}

Vec3 to_outer = center_ - inside.origin;
double proj_outer = dot(to_outer, inside.direction);
double dist_outer_sq = dot(to_outer, to_outer) - proj_outer *
proj_outer;
double radius_sq = config_.radius * config_.radius;
if (dist_outer_sq > radius_sq) {
    return inside;
}

double thc_outer = std::sqrt(radius_sq - dist_outer_sq);
double t_exit = proj_outer + thc_outer;
Vec3 exit_point = inside.at(t_exit);
Vec3 exit_normal = normalize(exit_point - center_);
Ray outside = refract(inside, exit_point, exit_normal,
                      config_.refractive_index, 1.0, total);
if (total) {

```

```

        Vec3 dir = normalize(inside.direction);
        double cos_i = -dot(exit_normal, dir);
        double fresnel_r = fresnel_schlick(cos_i, config_.
refractive_index, 1.0);
        double reflected_intensity = inside.intensity * fresnel_r;
        outside = Ray{exit_point + exit_normal * 1e-4,
                        reflect_dir(inside.direction, exit_normal),
                        reflected_intensity, inside.color};
    }

    double absorption = std::max(0.0, 1.0 - config_.transparency);
    outside.intensity *= (1.0 - absorption);
    outside.color = outside.color * config_.color;

    if (outside.intensity < 1e-5) {
        return std::nullopt;
    }

    if (config_.scatter_factor > 0.0) {
        std::normal_distribution<double> dist(0.0, config_.
scatter_factor);
        Vec3 jitter(dist(rng), dist(rng), dist(rng));
        Vec3 new_dir = normalize(outside.direction + jitter);
        outside.direction = new_dir;
    }

    return outside;
}

```

A.4 Ядро трассировки лучей

Листинг A.4 — Метод `Renderer::trace_ray` — ядро алгоритма трассировки лучей

```

void Renderer::trace_ray(const Ray& ray,
                        const LightConfig& source,
                        const std::vector<LightConfig>& lights,
                        const WallConfig& wall,
                        const std::vector<SphereConfig>& spheres,
                        const RenderSettings& settings,
                        ImageBuffer& wall_buffer,

```



```

        std::vector<ImageBuffer>& sphere_buffers)
{
    if (ray.intensity < 1e-5) {
        return;
    }

    double closest_t = 1e10;
    bool hit_object = false;
    int hit_type = 0;
    int hit_index = 0;
    Vec3 hit_point;
    Vec3 hit_normal;

    for (size_t i = 0; i < spheres.size(); ++i) {
        Vec3 point, normal;
        double t = 0.0;
        if (intersect_sphere(ray, spheres[i], point, normal, t)) {
            if (t < closest_t && t > 1e-6) {
                closest_t = t;
                hit_object = true;
                hit_type = 1;
                hit_index = static_cast<int>(i);
                hit_point = point;
                hit_normal = normal;
            }
        }
    }

    Vec3 wall_point;
    double wall_t = 0.0;
    bool hit_wall = intersect_wall(ray, wall, wall_point, wall_t);
    if (hit_wall && wall_t < closest_t && wall_t > 1e-6) {
        closest_t = wall_t;
        hit_object = true;
        hit_type = 0;
        hit_point = wall_point;
        hit_normal = WALL_NORMAL;
    }

    if (!hit_object) {
        return;
    }
}

```

```

    }

    double lambert = std::max(0.0, dot(hit_normal, ray.direction *
-1.0));
    double final_intensity = ray.intensity * lambert;

    if (hit_type == 0) {
        double norm_x = (hit_point.x + wall.width * 0.5) / wall.
width;
        int pixel_x = static_cast<int>(norm_x * wall.resolution_x);
        double norm_y = (hit_point.y + wall.height * 0.5) / wall.
height;
        int pixel_y = static_cast<int>((1.0 - norm_y) * wall.
resolution_y);
        wall_buffer.add_pixel(pixel_x, pixel_y, ray.color,
final_intensity);
    } else if (hit_type == 1) {
        const auto& sphere = spheres[hit_index];
        double u, v;
        sphere_point_to_uv(hit_point, sphere, u, v);
        int pixel_x = static_cast<int>(u * sphere.resolution);
        int pixel_y = static_cast<int>((1.0 - v) * sphere.
resolution);
        if (hit_index < static_cast<int>(sphere_buffers.size())) {
            sphere_buffers[hit_index].add_pixel(pixel_x, pixel_y,
ray.color, final_intensity);
        }
    }
}
}

```

ПРИЛОЖЕНИЕ Б

Презентация.